

Q&A cheat sheet — skills vs. agents

Mercatus, 2026-06-09. What's the difference, why both, and what's actually in this workflow.

The one-sentence difference

A **skill** is a Markdown checklist you trigger with a slash command, run by the Claude you're already talking to. An **agent** is a fresh Claude instance you spawn with its own focused context and persona, to which you delegate a self-contained task.

Skills

What: a folder of plain-Markdown files at `.claude/skills/<name>/SKILL.md` that defines a multi-step procedure or checklist.

How invoked: the user types `/<skill-name>`.

Where it runs: in the main conversation, with the same Claude session you've been talking to.

Best for: repeatable workflows, codified protocols, quick shortcuts. "How to compile this paper." "How to validate the bibliography." "How to commit, PR, and merge."

Cost: tokens consume the main conversation's context window.

Agents

What: a folder of Markdown files at `.claude/agents/<name>.md` that defines a sub-agent with a persona, a tool allowlist, and a system prompt.

How invoked: the main Claude calls the **Agent** tool, which spawns a new Claude instance with an isolated context.

Where it runs: a separate conversation. The sub-agent does its work, returns one final message, then disappears.

Best for: focused single-purpose review, deep judgement calls, parallel work, anything that would pollute the main context with raw output.

Cost: cheaper to the main context, but each agent run is its own Claude session.

Why you need both (four reasons)

- **Context hygiene.** A bibliography scan reads every `.tex` file in the manuscript. If that ran in your main session, your context would fill with raw bib entries. As an agent it stays clean.
- **Persona isolation.** The proofreader and the domain reviewer want different headspaces. Each agent gets its own system prompt and behaves like a different specialist. Skills don't change persona.
- **Parallelism.** You can spawn three agents on the same section simultaneously. Skills run one at a time in the main thread.
- **Workflow vs. judgement.** `/compile-latex` is three deterministic shell commands; that wants a skill. "Did I represent Krugman accurately?" is judgement; that wants an agent.

Agents currently in this workflow (4)

Agent	What it does
proofreader	Grammar, typos, consistency, LaTeX hygiene visible from text. Read-only.
domain-reviewer	Substantive review — derivation correctness, assumption sufficiency, citation fidelity, code-theory alignment, logical consistency.
r-reviewer	R code quality, reproducibility, figure-generation patterns, project theme compliance.
verifier	End-to-end verification — does the manuscript compile, do the R scripts run, are the output files non-empty.

Skills currently in this workflow (12)

Skill	What it does
<i>Build / verify</i>	
<code>/compile-latex</code>	3-pass XeLaTeX + bibtex on <code>Paper/main.tex</code> .
<code>/validate-bib</code>	Cross-reference <code>\cite{}</code> keys against the <code>.bib</code> file.
<code>/commit</code>	Stage, commit, push, open PR, merge.
<i>Review (skills that invoke the agents above)</i>	
<code>/proofread</code>	Runs the proofreading protocol on the manuscript.
<code>/review-paper</code>	Comprehensive manuscript review — argument, methods, citations, referee objections.
<code>/review-r</code>	R code review protocol on scripts.
<i>Research / writing workflows</i>	
<code>/lit-review</code>	Structured literature search and synthesis with citation extraction.
<code>/research-ideation</code>	Generate testable hypotheses and empirical strategies from a topic.
<code>/interview-me</code>	Interactive interview to formalise a research idea into a spec.
<code>/data-analysis</code>	End-to-end R analysis from exploration through publication-ready tables.
<i>Project meta</i>	
<code>/context-status</code>	Show current context usage and session health.
<code>/learn</code>	Extract reusable knowledge from this session into a new skill.

Two concrete examples of skills + agents working together

Example 1: gating a section before commit.

1. Finish drafting `Paper/sections/09_growth.tex`.
2. In the main session: type `/review-paper`.
3. The skill spawns the proofreader and domain-reviewer agents in parallel against the section.
4. Each agent runs in its own context and returns a Markdown report to the main session.
5. I read the two reports, decide which findings to act on, and edit accordingly.
6. Type `/validate-bib` (skill, runs in main context — it's just a regex sweep).
7. Type `/compile-latex` (skill, runs the three shell commands).
8. Type `/commit` (skill, stages and pushes).

The two agents catch judgement-calls; the four skills handle deterministic workflow.

Example 2: when you'd use an agent vs. a skill for the same task.

“Check the bibliography.”

- **As a skill** (`/validate-bib`): regex extracts every `\cite{}` key, compares to the `.bib`, reports missing/unused entries. Deterministic. Runs in 5 seconds in the main context.
- **As an agent** (substance-review): reads the `.bib` entries, checks whether each citation in the text accurately represents what the cited work actually argues. Judgement-heavy. Wants its own context.

Same surface task, two different jobs. Skill catches the mechanical gap (`Smith1776_wealth` is in the text but not in the bib). Agent catches the substantive gap (you cited Krugman 1979 for a claim Krugman 1991 actually makes).

Three verbal anchors

- *Skill in one sentence*: “A slash command that runs a checklist I wrote down, in the conversation I’m already in.”
- *Agent in one sentence*: “A fresh Claude with its own persona and context that I delegate a focused task to, and which sends one report back.”
- *Why both*: “Skills are deterministic workflow. Agents are judgement that wants its own headspace. The first stops me typing the same three commands every time. The second catches things a single Claude session would miss because it’s juggling too much.”